Code: Shared TypeScript types (types.ts)

Project reference: STN22_Thoracic belt for respiratory rate measurement

Authors: Hakkou Dounia

File header

```
<!--
  Description: Central TypeScript type definitions shared across
  all Svelte components in the RespiraMed PWA.
  Exporting types from a dedicated file ensures a single source of truth
  for data structures and prevents duplication across components.

  Project reference: STN22_Thoracic belt FR
  Authors: Hakkou Dounia
  Date: 2025-2026
  © SIS TEAM NANCY
-->
```

Patient type definition

```
// Patient describes the data structure of a patient record in the application.
// Fields marked with '?' are optional because they are absent before the record
// is persisted by the backend (id, created_at) or have a default value (sexe).
export type Patient = {

  // Unique identifier generated by the MySQL backend on insertion.
  // Optional here because the object may not yet exist in the database
  // (e.g. when building the payload for a POST /patients request).
  id?: string

  // Mandatory nominative fields, collected through PatientForm.svelte
  nom: string          // Family name
  prenom: string       // First name

  // Mandatory physiological fields used for patient identification
  age: number          // Age in full years

  // Literal union type: only 'M', 'F', or 'Autre' are accepted.
  // This prevents invalid values at compile time (stronger than plain string).
  sexe: "M" | "F" | "Autre"

  taille: number       // Height in centimetres
  poids: number        // Weight in kilograms
  telephone: string    // Contact phone number

  // ISO 8601 timestamp set by the server at creation time.
  // Optional for the same reason as id: absent before the record is saved.
  created_at?: string
}

// Note: api.ts also defines a Patient interface with slightly different constraints:
// - id and created_at are required (not optional) in api.ts because those
//   fields are always present in backend responses.
// - sexe is typed as plain string in api.ts (no union restriction).
// Use this type (types.ts) in form components and types that pre-date persistence.
// Use the api.ts interface when working with data returned from the REST API.
```